UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

DATA STRUCTURES – CS163

TREAP

Class - Group:	Class 25A01 – Group 6
Students:	Bui Viet Thanh – 25125065
	Nguyen Le Tran – 25125039
	Le Quynh Anh – 25125071
	Dinh Nguyen Hong Anh – 25125078
Instructors:	PhD Dinh Ba Tien
	MSc Truong Phuoc Loc
	MSc Ho Tuan Thanh

Contents

1 Introduction and History **1**
- 1.1 Definition and Background . . . 1
- 1.2 Historical note . . . 1

2 Core Data Structure **1**
- 2.1 Treap invariants . . . 1
- 2.2 Random priorities and expected height . . . 2

3 Main Algorithms **2**
- 3.1 Split and Merge . . . 2
- 3.2 Pseudo-code: split . . . 3
- 3.3 Pseudo-code: merge . . . 3
- 3.4 Insertion (split + merge) . . . 3
- 3.5 Deletion (split + merge) . . . 4
- 3.6 Searching . . . 4

4 Step-by-Step Demonstration **4**
- 4.1 Demo A: inserting one key using split–merge . . . 4
- 4.2 Demo B: implicit treap (sequence operations) . . . 5

5 Implementation Notes and Complexity **5**
- 5.1 Implementation choices . . . 5
- 5.2 Randomness and reproducibility . . . 5
- 5.3 Time complexity . . . 6
- 5.4 Space complexity . . . 6
- 5.5 Practical engineering notes . . . 6

6 Applications **6**
- 6.1 Ordered set / map (dictionary) . . . 6
- 6.2 Order statistics and custom queries . . . 7
- 6.3 Dynamic sequence representation (implicit treap) . . . 7
- 6.4 Flexible data structure for custom problems . . . 7

7 Datasets **7**
- 7.1 Synthetic Data . . . 7
- 7.2 Google Web Traffic Time Series . . . 7

8 Comparison of Runtime and Memory Usage **8**
- 8.1 Treap versus Array (Random Data) . . . 8
- 8.2 Treap versus Normal BST (Increasing Trend) . . . 8

9 Assignments **9**
9.1 CSES 1144 - Salary Queries 9
9.2 ORDERSET - Order statistic set 9
9.3 KQUERY - K-query 9
9.4 CSES 2072 - Cut and Paste 9

10 Conclusion **10**

11 References **10**

1 Introduction and History

1.1 Definition and Background

A *treap* is a randomized binary search tree in which each node stores two values: (1) a **key**, used to maintain the binary-search-tree (BST) ordering, and (2) a **priority**, chosen randomly at insertion time and used to maintain a heap ordering.

Formally, a treap satisfies simultaneously:

- the **BST property** with respect to keys, and
- the **heap property** with respect to priorities.

For any fixed set of (key, priority) pairs, these two properties uniquely determine the structure of the tree. When priorities are independent and uniformly random, the resulting treap has the same distribution as a binary search tree constructed by inserting keys in random order.

Balanced search trees such as AVL trees or Red–Black trees guarantee $O(\log n)$ height by enforcing strict structural constraints. In contrast, a treap achieves balance *in expectation* through randomization, without maintaining explicit balance factors or coloring rules. As a consequence, search, insertion, and deletion run in expected $O(\log n)$ time while keeping the implementation relatively simple. [1, 2]

1.2 Historical note

Treaps were introduced by Seidel and Aragon (1989) under the name *randomized search trees*. The key observation is that if each key receives an independent random priority, then the resulting tree has the same distribution as a BST produced by inserting keys in a uniformly random order. This gives logarithmic height in expectation without maintaining strict balance invariants (such as AVL balance factors or Red–Black coloring rules), making treaps attractive in both theory and practice. [2, 1]

2 Core Data Structure

2.1 Treap invariants

A treap stores each element as a node containing a **key** (for BST ordering), a **priority** (assigned randomly at insertion time), pointers to **left/right** children, and optionally **augmented fields** (e.g., subtree size, subtree sum) used to support advanced queries.

A treap must satisfy two invariants:

1. **BST property (by key):** all keys in the left subtree are smaller, and all keys in the right subtree are larger.
2. **Heap property (by priority):** priorities obey a heap rule (either max-heap or min-heap), so each node has priority at least as extreme as its children.

These two properties together ensure that the in-order traversal returns keys in sorted order, while the heap rule drives the expected balance of the structure. [1, 4]

2.2 Random priorities and expected height

If priorities are independent and uniformly random, then treaps behave like randomly built BSTs. As a result, the height is $O(\log n)$ *in expectation*, which yields expected $O(\log n)$ time for search, insertion, and deletion. While the worst case is still $O(n)$ (rare but possible), the randomized construction makes such degenerate shapes unlikely in typical use. [1, 4]

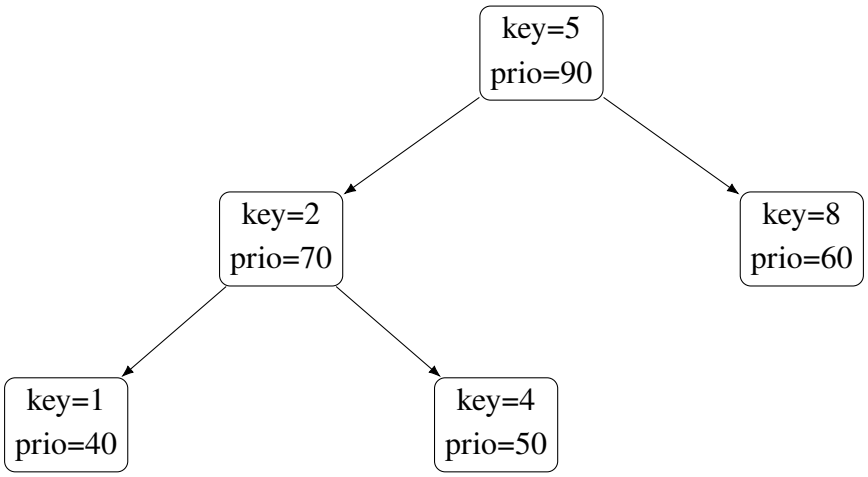

Figure 1: Example treap: BST-ordered by `key` and heap-ordered by `priority`.

3 Main Algorithms

3.1 Split and Merge

Many treap implementations are built around two core primitives:

Split: given a treap T and a key x, split T into (L, R) such that all keys in L are $\leq x$ and all keys in R are $> x$.

Merge: given treaps L and R where every key in L is $\leq$ every key in R, merge them into one treap while preserving both BST order and heap order.

These primitives make insertion and deletion compact, and they extend naturally to more advanced operations such as splitting by position in implicit treaps. [2, 3]

3.2 Pseudo-code: split

Algorithm 1 Split(T, x) (BST by key; heap by priority)

```
procedure SPLIT(T, x)
    if T = null then
        return (null, null)
    if T.key ≤ x then
        (A, B) ← SPLIT(T.right, x)
        T.right ← A
        return (T, B)
    else
        (A, B) ← SPLIT(T.left, x)
        T.left ← B
        return (A, T)
```

3.3 Pseudo-code: merge

Algorithm 2 Merge(L, R) where $\max(L.key) \leq \min(R.key)$

```
procedure MERGE(L, R)
    if L = null then
        return R
    if R = null then
        return L
    if L.prio > R.prio then                    ▷ max-heap example
        L.right ← MERGE(L.right, R)
        return L
    else
        R.left ← MERGE(L, R.left)
        return R
```

3.4 Insertion (split + merge)

To insert a new key k with a random priority, we split the current treap T into (L, R) by k, create a new node $N = (k, \text{random_prio})$, and merge back as:

$$T \leftarrow \text{Merge}\big(\text{Merge}(L, N), R\big).$$

This keeps BST order by construction (all of L is $\leq k$ and all of R is $> k$), while heap order is restored by `Merge`. [2, 3]

3.5 Deletion (split + merge)

To delete a single occurrence of a key k, the split–merge approach must be used with care. The basic two-split method isolates all nodes with key k, which is suitable only when keys are unique or when all duplicates should be removed.

To delete exactly one occurrence, we instead locate a node with key k following the BST property, and remove it by merging its left and right subtrees. Specifically, once a node with key k is found, it is replaced by:

$$T \leftarrow \text{Merge}(T.left, T.right).$$

This operation preserves both the BST and heap properties of the treap.

This approach removes exactly one node, avoids affecting other duplicates, and maintains correctness of any augmented fields by updating them during the merge operation. [2]

3.6 Searching

Searching is identical to a standard BST search: compare keys and move left/right until a match is found or a null pointer is reached. Because expected height is $O(\log n)$ under random priorities, expected search time is $O(\log n)$. [1, 4]

4 Step-by-Step Demonstration

4.1 Demo A: inserting one key using split–merge

Assume we have a treap containing keys $\{1, 2, 4, 7\}$ and we want to insert $k = 5$. We perform:

1. **Split by 5:** $(L, R) = \text{Split}(T, 5)$ so L contains $\{1, 2, 4\}$ and R contains $\{7\}$.
2. **Create node:** $N = (5, p)$ with random priority p.
3. **Merge back:** $T \leftarrow \text{Merge}(\text{Merge}(L, N), R)$.

Intuitively, the split isolates where key 5 belongs in sorted order, and merge reinserts the new node while maintaining heap order by priority. [2]

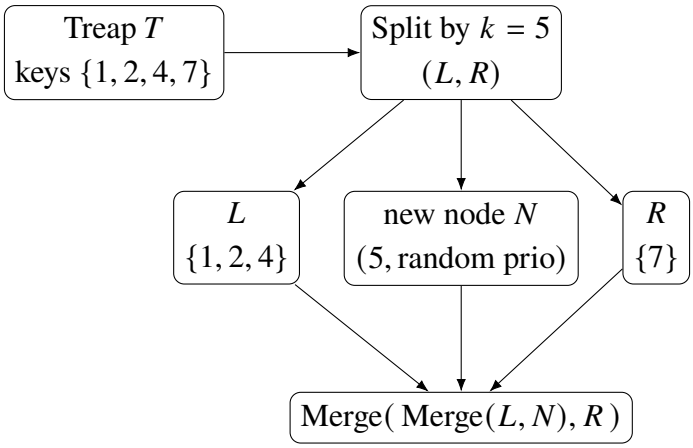

Figure 2: Insertion workflow using split–merge.

4.2 Demo B: implicit treap (sequence operations)

An **implicit treap** stores a sequence (like a dynamic array). Instead of using the stored key to guide BST order, the 'key" is the node's *in-order index*, computed using subtree sizes. This allows splitting by position (first k elements vs. the rest) and then merging sequences back together.

A common sequence-editing operation is **cut-and-paste**. To move a subarray $[l, r]$:

1. Split into `prefix` ($[1, l-1]$) and `rest` ($[l, n]$).
2. Split `rest` into `middle` ($[l, r]$) and `suffix` ($[r+1, n]$).
3. Merge in a new order, e.g. `middle` + `prefix` + `suffix`.

With lazy propagation, the same structure supports range reverse and other range updates efficiently. [3]

5 Implementation Notes and Complexity

5.1 Implementation choices

There are two widely used implementation styles. In a **rotation-based** treap, we insert like a BST and then rotate the node upward (or downward) to restore the heap property, similar in spirit to heap fixing. In a **split–merge-based** treap, `Split` and `Merge` are the foundation, and insertion/deletion reduce to a few lines of code that are easy to reason about. Both are correct; split–merge is often preferred for clean support of augmented fields and sequence variants. [4, 2, 3]

5.2 Randomness and reproducibility

Because expected balance depends on random priorities, practical implementations should use a robust PRNG, optionally fix a seed during testing for reproducibility, and choose a large priority range (e.g., 64-bit) to minimize collisions. If collisions occur, a deterministic tie-break (e.g comparing memory address or insertion order) can preserve the heap property consistently. [1, 2]

5.3 Time complexity

Expected time for search, insertion, and deletion is $O(\log n)$ because the expected height of the treap is $O(\log n)$ under random priorities. The worst case remains $O(n)$ if the tree becomes highly unbalanced, but this is unlikely when priorities are independent and uniformly distributed. [1, 4]

5.4 Space complexity

A treap stores one node per key, plus constant metadata (priority and pointers). Therefore space usage is $O(n)$, and augmented fields add only constant extra space per node. [1]

Table 1: Typical complexity summary for treap operations (expected vs. worst-case).

Operation	**Expected**	**Worst-case**
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Space	$O(n)$	$O(n)$

5.5 Practical engineering notes

Treaps are often implemented recursively because split/merge are naturally recursive, but recursion depth could become large in a worst-case tree. In competitive programming and many systems contexts this is acceptable; otherwise, iterative variants or tail-recursion optimizations can be considered. Augmentation is a major practical advantage: by maintaining subtree size, sum, or minimum/maximum values, a treap can answer rank and range queries efficiently, and implicit treaps can support range operations with lazy tags. [2, 3]

6 Applications

6.1 Ordered set / map (dictionary)

Treaps can be used to implement ordered sets or maps, supporting operations such as insertion, deletion, and search in expected $O(\log n)$ time. While their asymptotic complexity is comparable to more strictly balanced trees such as AVL or Red–Black trees, their main advantage lies in their simplicity. Treaps require fewer structural rules and are significantly easier to implement from scratch, making them attractive when a lightweight yet efficient ordered container is needed. This simplicity can also lead to practical performance benefits, as shorter and cleaner code is often easier for compilers to optimize. [1, 2]

6.2 Order statistics and custom queries

A key advantage of treaps is their flexibility for customization. By augmenting each node with additional information (such as subtree size), treaps can efficiently support operations like finding the rank of an element or selecting the k-th smallest element. Such functionality is not directly available in standard library structures like `std::set`. This makes treaps particularly useful when more specialized queries are required, as developers can extend the data structure with minimal additional code. [2]

6.3 Dynamic sequence representation (implicit treap)

Treaps can be adapted to represent sequences by treating the in-order position as the key (implicit treap). This allows efficient manipulation of arrays or sequences, including insertion, deletion, and rearrangement of segments. Although similar functionality can be achieved with other balanced trees, treaps provide a concise and flexible way to implement these operations, making them especially popular in competitive programming and algorithmic problem solving. [3]

6.4 Flexible data structure for custom problems

Treaps are particularly useful in situations where standard library data structures are insufficient. When a problem requires combining multiple operations - such as maintaining order, supporting custom queries, or modifying structure dynamically - treaps allow developers to design exactly what is needed. Their implementation is relatively short (often within tens of lines), making them a practical choice when building tailored solutions. In such cases, the benefit of treaps comes not from superior theoretical complexity, but from their adaptability and ease of implementation. [2]

7 Datasets

7.1 Synthetic Data

We generated simulated query streams consisting of 50% Find, 40% Insert, and 10% Delete operations. Two data variants were used: totally random data (to benchmark against Arrays) and increasing-trend data (to benchmark against standard BSTs).

7.2 Google Web Traffic Time Series

For real-world testing, we used daily traffic data from 144,000 Wikipedia articles recorded from July to December 2015. The task was to continuously maintain and query the median value dynamically as new data arrived in a stream.

8 Comparison of Runtime and Memory Usage

8.1 Treap versus Array (Random Data)

- **Time Complexity:** The Array exhibits an $O(N^2)$ overall runtime for N queries because finding and deleting elements requires $O(N)$ time per operation. In contrast, the Treap maintains an expected $O(\log N)$ time per operation, resulting in a highly efficient, near-flat execution curve.
- **Space Complexity:** While both structures operate in $O(N)$ space, the Array has a much smaller constant factor due to contiguous storage. The Treap consumes more absolute memory because of the structural overhead of storing child pointers and priority values within each node.

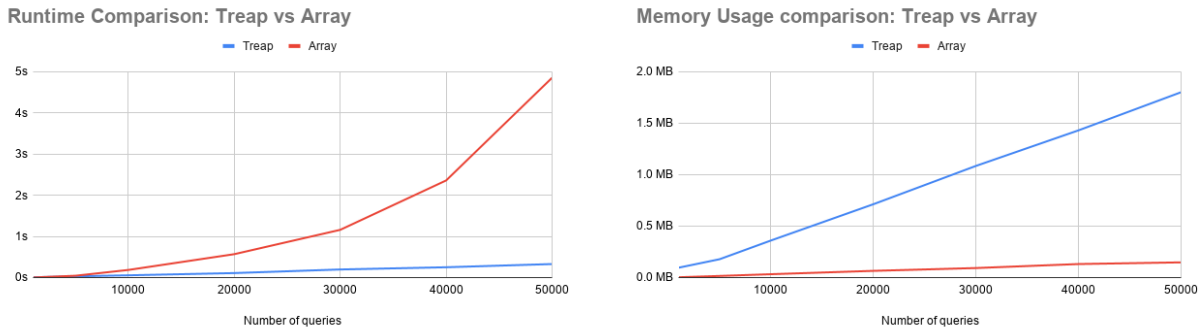

Figure 3: Performance metrics of Treap versus naive Array under purely random operations.

8.2 Treap versus Normal BST (Increasing Trend)

- **Time Complexity:** Sequentially increasing data represents the worst-case scenario for a standard BST, causing it to degenerate into a linear linked list with $O(N)$ search and insert time. The Treap circumvents this structural degradation through its randomized priorities, ensuring the tree remains balanced with an expected $O(\log N)$ height.
- **Space Complexity:** Both trees require exactly $O(N)$ space. The Treap's slight memory increase over the normal BST is strictly due to the $O(1)$ extra space per node needed to store the random priority value. This minor constant-factor overhead is a necessary trade-off to guarantee logarithmic time complexity.

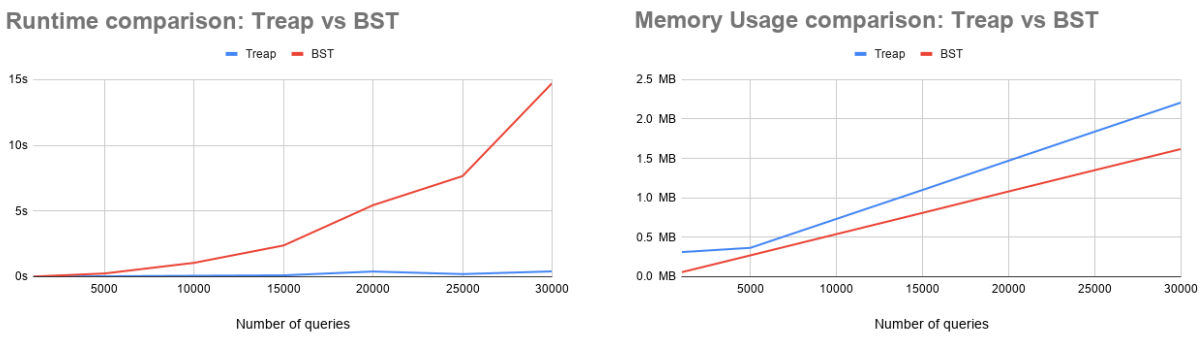

Figure 4: Performance metrics of Treap versus normal BST under sequentially increasing data.

9 Assignments

To demonstrate the practical utility and versatility of the Treap data structure, we analyzed and applied it to solve several advanced algorithmic problems from prominent competitive programming platforms.

9.1 CSES 1144 - Salary Queries

Problem Link: https://cses.fi/problemset/task/1144
Approach: Use a Treap to store the employees salaries. To change the employee salary, delete the old salary and insert the new one. For query of counting number of employees whose salary is between a and b, split the Treap at a and b and count the remaining elements.

9.2 ORDERSET - Order statistic set

Problem Link: https://www.spoj.com/problems/ORDERSET/
Approach: Use a Treap with subtree sizes. Answer K-TH queries by traversing the Treap based on subtree sizes. Compute `COUNT(x)` by recursively counting elements smaller than x, also using subtree sizes.

9.3 KQUERY - K-query

Problem Link: https://www.spoj.com/problems/KQUERY/
Approach: Sort the array elements and queries in decreasing order of array values and k. Use a Treap with subtree sizes to maintain the positions of elements greater than the current k. For each query, insert all elements with value greater than k into the Treap. Answer each query by counting the number of positions in range `[l, r]` using subtree sizes.

9.4 CSES 2072 - Cut and Paste

Problem Link: https://cses.fi/alon/task/2072
Approach: Use an Implicit Treap to store the string and do Lazy Propagation. When we need to

reverse the segment `[L, R]`, split that segment into an independent subtree, toggle a boolean flag `is_reversed = true` at the root node of that subtree, and then merge it back. A reversed node is updated by swapping its left and right children and push the `is_reversed` flag to them.

10 Conclusion

A treap is a randomized balanced BST that combines BST ordering by key with heap ordering by random priority. Its expected logarithmic height enables efficient search, insertion, and deletion, while keeping implementations relatively simple. With split and merge as core primitives, many operations become concise and composable, especially for augmented and implicit variants. As a result, treaps are widely useful: as ordered sets/maps, as order-statistic and range-query structures, and as dynamic sequence editors via implicit treaps. [1, 2, 3]

11 References

References

[1] Pat Morin. *Open Data Structures (in Java)* — Section 7.2: Treap: Randomized Binary Search Tree. Online textbook, accessed April 17, 2026. https://opendatastructures.org/ods-java/7_2_Treap_Randomized_Binary.html

[2] cp-algorithms (e-maxx). *Treap*. Online article, accessed April 17, 2026. https://cp-algorithms.com/data_structures/treap.html

[3] USACO Guide. *Treaps (Advanced)*. Online guide, accessed April 17, 2026. https://usaco.guide/adv/treaps?lang=cpp

[4] GeeksforGeeks. *Treap | A Randomized Binary Search Tree*. Online tutorial, accessed April 17, 2026. https://www.geeksforgeeks.org/dsa/treap-a-randomized-binary-search-tree/

[5] Felipe Vaiano Calderan. *An Analysis of Treap*. Federal University of São Paulo (UNIFESP), July 2022.

[6] H. S. Anand and S. S. Vinodchandra. *Association Rule Mining Using Treap*. International Journal of Machine Learning and Cybernetics, 2016. doi:10.1007/s13042-016-0546-7.